DS	**Session**	Queue	**Question Information**	Level 1 Challenge 55

Question description

Sathya is an DS expert training youngsters struggling in DS to make them better.

Sathya usually gives interesting problems to the youngsters to make them love the DS.

One such day Sathya provided to the youngsters to solve the task such that, insert an element in a Queue in FIFO order

Youngsters were lacking the idea to solve the problem.

Being an exciting youngster can you solve it?

Function Description

1. Define the maximum size of queue and initialize front and rear as -1.
2. In the main function we will initialize two variables that will store the data and the size of the queue.
3. Accept the data that we want to enter in a queue using a for loop.
4. After accepting the data use enqueue() function to insert the data in a queue.
5. In this function return queue id full , if the value of rear is equal to max-1.
6. Else increase the value of rear by 1.
7. After this insert the data that we have accepted earlier.
8. Now display the data of the queue using a disp() function.
9. In this function if the value of front is equal to -1 then it means list is empty.
10. Else initialize a for loop from front to rear and print the data that queue is holding.

Constraints:

0<size <100

0<data<1000

Input format:

First line indicates the size of the queue

Second line indicates the elements of the queue.

Output Format:

every lines indicates the enqueue of each elements

last line indicates the fine queued elements.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
5 6 3 1 2
```

EXPECTED OUTPUT

```
Enqueuing 5
5 Enqueuing 6
5 6 Enqueuing 3
5 6 3 Enqueuing 1
5 6 3 1 Enqueuing 2
5 6 3 1 2
```

Test Case 2

INPUT (STDIN)

```
8
1 8 6 4 9 5 3 5
```

EXPECTED OUTPUT

```
Enqueuing 1
1 Enqueuing 8
1 8 Enqueuing 6
1 8 6 Enqueuing 4
1 8 6 4 Enqueuing 9
1 8 6 4 9 Enqueuing 5
1 8 6 4 9 5 Enqueuing 3
1 8 6 4 9 5 3 Enqueuing 5
1 8 6 4 9 5 3 5
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void enqueue(int data)
```

Test Case 2

KEYWORD

```
for(i=front;i<=rear;i++)
```

Test Case 3

KEYWORD

```
enqueue(data);
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY

3

TOKEN COUNT

200

NLOC

44

Code

solution.c

```c
#include <stdio.h>

int queue[100];
int front = 0, rear = -1;

void enqueue(int data) {
    queue[++rear] = data;
}

void disp() {
    int i;
    for (i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    for (int i = 0; i < n; i++) {
        int data;
        scanf("%d", &data);
        if (i > 0) {
            disp();
        }
        printf("Enqueuing %d\n", data);
        enqueue(data);
    }
    disp();
    printf("\n");
    return 0;
}
```

Output

Problems Output Debug Console Terminal Ports powershell

```
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
5
5 6 3 1 2
Enqueuing 5
5 Enqueuing 6
5 6 Enqueuing 3
5 6 3 Enqueuing 1
5 6 3 1 Enqueuing 2
5 6 3 1 2
PS C:\C Programming>
```